

Document de maintenance — PertFlow

Guide pratique pour **reprendre et faire évoluer** PertFlow. À lire avec `conception.md`, qui explique l'architecture et *pourquoi* elle est ainsi. Ces deux documents, avec le manuel utilisateur, sont la mémoire du projet : ce qui n'y figure pas n'a pas à être cherché ailleurs.

1. Reprendre le projet en 2 minutes

- **Aucun build, aucune installation** pour utiliser l'outil : ouvrez `index.html` (ou `dist/pertflow.html`) par double-clic → il s'ouvre en `file://` dans le navigateur.
 - **Pour développer**, éditez `index.html`, `src/*.js`, `css/style.css` et rechargez la page.
 - Un serveur local (`npx serve` ou `python -m http.server`) n'est qu'un **confort de dev ponctuel** : il ne doit **jamais** devenir nécessaire (voir contraintes ci-dessous).
 - **Pour valider** une modification, en revanche, il y a une installation (Node + le Chromium de Playwright) : voir `tools/README.md`, puis `cd tools && npm test`.
-

2. Contraintes ABSOLUES (à ne jamais enfreindre)

Ces règles découlent du déploiement sur postes DSI verrouillés (ouverture `file://`) :

1. **Pas de modules ES6.** Interdits : `<script type="module">`, `import` / `export`. Le code est chargé par des `<script src>` classiques et vit dans le **scope global**.
2. **Pas de `fetch()` /XHR de fichiers locaux.** Lire les fichiers utilisateur avec `<input type="file">` + `FileReader`. (Vigilance particulière sur l'import/export.)
3. **Licence MIT uniquement.** Toute nouvelle bibliothèque doit être MIT (ou compatible) et **locale** (`lib/`), jamais via CDN. Préférer réutiliser `fflate/jsPDF/LiteGraph`.
4. **Pas de caractères accentués dans les identifiants ou valeurs de code** (les libellés d'affichage peuvent l'être).
5. **Commentaires auto-documentés systématiques** ; ne pas supprimer les commentaires existants hors lignes réellement modifiées.

Toute PR qui introduirait un module ES6, un `fetch` local ou une dépendance non-MIT casse le déploiement cible.

3. Pièges LiteGraph (déjà rencontrés — à connaître absolument)

Piège	À faire
Masquer la barre de titre d'un nœud	<code>MonNoeud.title_mode = LiteGraph.NO_TITLE</code> sur le constructeur . <code>flags.no_title</code> ne fait rien .
Rafraîchir le canvas	<code>LGraphCanvas.setDirty(fg, bg)</code> ; setDirtyCanvas est sur le graphe / le nœud , pas sur le canvas.
Slots d'entrée dynamiques des nœuds	Connecter deux liens au même slot 0 remplace le premier. En test, viser des slots successifs ou isoler.
Liens du nœud sélectionné forcés en blanc	LiteGraph met <code>#FFF</code> via <code>highlighted_links</code> → on le vide dans notre <code>onDrawBackground</code> pour que nos couleurs de lien priment.
Surcharger un comportement	Surcharger sur l' instance <code>LGraphCanvas</code> (<code>renderLink</code> , <code>getMenuOptions</code> , <code>getNodeMenuOptions</code>) plutôt que patcher <code>lib/litegraph.js</code> .
Rendu hors-écran (export)	Penser à <code>graph.detachCanvas(tmp)</code> après coup (sinon le canvas temporaire reste dans <code>list_of_graphcanvas</code>).
<code><datalist></code> natif	Inadapté au « choisir parmi les valeurs existantes » (masqué par <code>autocomplete=off</code> sous Firefox ; filtré par la valeur courante sous Edge/Chrome). Utiliser le menu déroulant custom (<code>buildCombobox</code>).

4. Comment ajouter...

...un réglage de projet

1. Ajouter le champ dans le dialogue **Paramètres** (`index.html`).
2. Lire/écrire dans `openSettings` / `saveSettings` (`ui.js`).
3. **Sérialiser** dans `storage.js` (côté `pertSerializeProject` **et** `pertApplyProject` , avec une **valeur par défaut robuste** pour les anciens `.pert`).
4. **Restaurer** dans `history.js` (undo).
5. Si le réglage a un effet visuel immédiat, l'appliquer dans `saveSettings` et au chargement.

(Exemple récent : `meta.link_mode` en Session 10.)

...une propriété de nœud

1. La déclarer dans `this.properties` du constructeur (`nodes.js`), **avec sa valeur par défaut** et un commentaire disant à quoi elle sert.
2. **Ne rien faire pour la sérialisation** : `node.properties` est sérialisé nativement par LiteGraph, et `configure()` **fusionne** les propriétés du fichier sur celles du nœud neuf — un `.pert` antérieur récupère donc les défauts du constructeur. **Aucune migration** n'est nécessaire, et il ne faut pas en écrire.
3. L'exposer dans le panneau (`showProperties` , `ui.js`). Se demander si elle appartient à la **planification** ou aux fonctions secondaires : voir la règle d'ordre du panneau au §7.
4. Si sa valeur peut être **déduite** d'une autre (cas de la charge : ETP $\rightleftharpoons$ heures), écrire un **accesseur** dans `pert_engine.js` et faire passer **tous** les consommateurs par lui — jamais `node.properties.x` en direct ailleurs. Sans quoi les exports et la synthèse liront une valeur périmée sans que rien ne le signale.

...une colonne à un export

- **En FIN de schéma, jamais intercalée.** Les dépouillements utilisateurs (tableaux croisés, formules) repèrent leurs colonnes par **position** : une insertion au milieu les casse tous en silence. C'est la seule règle vraiment intangible du CSV.
- Côté test, **repérer la colonne par son en-tête**, jamais par un index figé — sinon le prochain ajout fera échouer des tests sans rapport avec ce qu'ils protègent (c'est arrivé deux fois).

...un format d'export

1. Créer `src/export_<format>.js` qui produit le contenu (réutiliser `pertXlsxBuild` pour un Excel, `pertScheduleModel` pour du temps/charge/liens) et télécharge via `pertDownloadBlob` .
2. Appeler `pertRegisterExportFormat({ id, icon, label, desc, order, run })` en fin de fichier.
3. Déclarer le `<script src>` dans `index.html` . Le bundle le reprend seul (`build-bundle.js` inline tout `<script src>` qu'il y trouve) — rien à déclarer ailleurs.
4. **Inscrire le format dans `tools/smoke-s9.js`** , à sa place dans la liste. Cette liste est volontairement positionnelle : l'ordre d'apparition dans la fenêtre (champ `order`) n'est vérifié que là.
5. Si le format produit un **fichier binaire**, le test doit vérifier son contenu, pas seulement ses premiers octets : un `.xlsx` à qui il manque une relation interne s'ouvre **sans erreur**, sur un document vide. Relire le zip avec `fflate.unzipSync` **dans la page** (fflate y est déjà chargé) plutôt que d'ajouter une dépendance à `tools/` .

...un type de nœud

Définir le type dans `nodes.js` (rendu custom, `title_mode` , slots), l'enregistrer auprès de LiteGraph, l'intégrer au moteur si pertinent (`pert_engine.js`) et à la sérialisation.

5. Outillage de validation (`tools/`)

Le mode d'emploi complet est dans `tools/README.md` — prérequis, lancement, jeux d'essai, conventions d'écriture d'un test. En résumé :

```
cd tools && npm install && npx playwright install chromium # une fois
npm test # 31 tests, ~90 s
```

- **Suite smoke** : chaque test pilote l'application dans un vrai Chromium ouvert en `file://` — le mode de déploiement cible — clique les vrais boutons, et vérifie ce qui est affiché ou exporté, erreurs console comprises. `run-smokes.js` les enchaîne et rend un compte rendu unique.
- **Jeux d'essai dans `test_cases/`**, versionnés en **liste blanche** (tout ignoré, chaque fichier réautorisé nommément). Aucun planning CPERT réel n'y figure — ce sont des documents d'entreprise : l'import CPERT est testé sur un classeur **fabriqué** par `tools/make-cpert-fixture.js`, conçu pour exercer chaque règle de lecture. Si vous disposez d'un vrai CPERT, le déposer ici ajoute une non-régression supplémentaire — cf. `tools/README.md`.
- **Captures d'écran** : les `doc-shots*.js` (un par chantier illustré) alimentent `docs/images/manuel/` (versionné), les `shots-*.js` produisent des captures de **relecture** dans `/tmp`, `screenshot.js` fait l'unitaire. Un script de capture qui recadre le panneau doit viser son **dernier bloc** (`lastElementChild`) et non un champ nommé : l'ordre des champs évolue, et une capture qui coupe juste avant ce qu'elle illustre ne se voit pas au moment du build.
- **`check-bundle.js`** vérifie le bundle livré, pas les sources — à lancer après un build.

La suite doit passer **avant** toute clôture de session : c'est le seul filet du projet.

Documentation : Markdown (base) + HTML autonome + PDF

Les documents (`docs/*.md`) sont la **source**. Pour **chaque** document, on génère aussi une **version HTML autonome** (images embarquées en data-URI, consultable hors ligne d'un double-clic) et un **PDF**. Ces sorties `docs/*.html` et `docs/*.pdf` sont **versionnées** (livrables).

- **Régénérer** après toute modification d'un `.md` : `node tools/build-docs.js`.
- Chaîne : `tools/_md2html.py` (python-markdown → HTML autonome + CSS d'impression, images en data-URI) puis Chromium headless (Playwright) → PDF A4.
- Prérequis : `pip install markdown` + le Chromium de Playwright.
- Captures du manuel : `node tools/doc-shots.js` (régénère `docs/images/manuel/`).

Règle : à chaque évolution d'un document, régénérer HTML **et** PDF, et les committer avec le `.md`.

6. Git, versionnage et rituel de fin de session

- **Branche par sujet** — `evo/<sujet>` pour une évolution fonctionnelle, `fix/<sujet>` pour un correctif, `chore/<sujet>` pour l'outillage ou la doc —, merge **no-ff** sur `main`, **tag** en fin de session. Le sujet du merge s'écrit `merge <branche> → main (<résumé>, N/N)`, le `N/N` étant le résultat de la suite de tests.
-  **Numérotation des tags décalée** : `vN` $\neq$ `SN` (la Session 2.5 et plusieurs lots de correctifs ont consommé des tags). Se fier à l'historique des tags, pas au numéro de session. Les correctifs successifs sur une session déjà taguée utilisent un **schéma patch** `vX.Y.Z`.
- **Format de commit** : trois paragraphes (résumé bref / description fonctionnelle avec le *pourquoi* / liste des fichiers et changements). Préfixe `[Plugin]` si LaBotBox/Simulia (sans objet ici).

Rituel de fin de session (obligatoire)

À chaque clôture, **avant** le commit final :

1. **Faire passer la suite** : `cd tools && npm test` (attendu : 31/31).
2. **Mettre à jour la documentation** touchée — les `.md` de `docs/`, les **captures** qu'une évolution d'IHM a périmées (`node tools/doc-shots-*.js`), les versions HTML/PDF (`node tools/build-docs.js`) et les **notes de version** — **avant** le push.
3. **Régénérer le bundle** avec le tag de la session : `node scripts/build-bundle.js --tag vX.Y`, puis le vérifier : `node tools/check-bundle.js vX.Y`.
4. **Committer + pousser le bundle** (`dist/pertflow.html`, **versionné**) avec le reste.
5. Le bundle embarque le bouton « **À propos** » (© Stéphane Guichard, licence MIT, date de génération et tag) : ces valeurs sont injectées par le build dans `window.PERTFLOW_BUILD` — **ne jamais les coder en dur**.
6. **Publier la release GitHub**, une fois le tag poussé — c'est ce qui met l'outil à disposition sans avoir à récupérer tout le dépôt :

```
bash node scripts/make-release.js --tag vX.Y # → dist/release/pertflow_vX_Y.zip gh
release create vX.Y dist/release/pertflow_vX_Y.zip \ --title "PertFlow vX.Y – <titre
des notes de version>" \ --notes-file <extrait de docs/release-notes.md pour cette
version>
```

- L'archive contient **l'application, le manuel PDF, les notes de version et un LISEZ-MOI** — rien d'autre : c'est une livraison, pas un miroir du dépôt.

- `make-release.js` **refuse d'écrire** si le bundle ne porte pas le tag demandé, ou si `docs/release-notes.md` n'a pas de section pour cette version. Les deux erreurs seraient indétectables à l'usage : le numéro affiché par « À propos » vient du bundle et non du nom de l'archive, et des notes muettes sur la version installée ne se remarquent pas non plus.
- `dist/release/` est **gitignoré** : l'archive doublerait le bundle, déjà versionné. Son hébergement, c'est GitHub Releases.
- Le **corps de la release reprend les notes de version** : une seule rédaction, orientée utilisateur, à un seul endroit — `docs/release-notes.md` , qui voyage aussi dans l'archive.

Ordre : finaliser code/doc → suite verte → régénérer bundle (`--tag`) → committer (source + bundle) → pousser → merger sur `main` → taguer → pousser le tag → **archive + release GitHub**.

7. Points de vigilance divers

- `test_cases/` est en liste blanche, ne pas la transformer en liste noire : le `.gitignore` ignore tout le répertoire et réautorise chaque fichier nommément, ce qui rend `git add -A` sans danger et laisse déposer un planning de travail sans risque de le publier. Avant d'ajouter un `!` sur un fichier Office, **inspecter son zip** : `docProps/` (auteur, classification), `xl/externalLinks/` (chemin complet des classeurs liés), `xl/comments*` (auteurs), `printerSettings` (serveur d'impression). Deux `.xlsx` aux cellules pourtant synthétiques ont été écartés pour cette seule raison le 30/07/2026.
 - `console.log` fonctionne ici (contrairement à l'intégration LaBotBox du dépôt jumeau) ; mais en `file://` l'utilisateur final n'a pas la console → passer par `showToast / showError` .
 - **Compatibilité navigateur** : cible Chrome/Edge/Firefox récents. Les contrôles « natifs » (`<datalist>` , `<option>` colorées) se comportent différemment d'un navigateur à l'autre → privilégier des composants DOM/CSS maison (pattern déjà en place).
 - Le `.pert` doit rester **rétro-compatible** : toute nouvelle clé `meta` a une valeur par défaut à la lecture (anciens fichiers). Pour une **propriété de nœud**, c'est acquis sans rien écrire — LiteGraph fusionne les propriétés du fichier sur celles du nœud neuf — à condition que le **défaut du constructeur soit le comportement historique** (règle appliquée pour `charge_mode` , dont le défaut `"etp"` laisse les anciens plannings au coût inchangé).
 - **Ordre des champs du panneau d'une Activité** (décision utilisateur, 31/07/2026) : un PERT sert d'abord à **planifier**. Viennent donc en tête libellé, durée, tâche anticipée, couleur, groupe, responsable, notes ; puis l'intertitre « **Suivi et coût** » et, derrière lui, l'avancement et la charge. **Toute nouveauté hors planification se range sous l'intertitre**, jamais au milieu du haut de panneau. Un test vérifie l'ordre complet : sans lui, la décision se déferait en silence.
-

8. Où trouver quoi

Je cherche...	Fichier
Le manuel utilisateur	docs/manuel-utilisateur.md
Ce qui a changé d'une version à l'autre	docs/release-notes.md
Comment valider une modification	tools/README.md
L'architecture et les choix techniques	docs/conception.md
Le calcul PERT	src/pert_engine.js
L'estimation de charge et de coût (ETP ⇌ heures)	src/pert_engine.js (bloc « Estimation de coût »)
Le rendu des nœuds / liens	src/nodes.js , src/link_routing.js
Le panneau, les dialogues, le filtre, la barre de statut	src/ui.js
Les décors de fond (repère T0, trame calendaire)	src/t0_marker.js , src/time_grid.js
Les imports (CPERT .xslm , .pert)	src/import.js , import_excel.js , import_pert.js
Les exports	src/export*.js
La sérialisation / l'undo / l'autosave	src/storage.js , history.js , autosave.js
Les fenêtres de rapport (synthèse, suivi d'avancement)	src/synthesis.js , src/suivi.js
La fabrication du bundle et de l'archive de livraison	scripts/build-bundle.js , scripts/make-release.js